

Proyecto 2

Ahorcado: juego electrónico FPGA / PC por enlace serial

Profesor

Dr.-Ing. Jeferson González-Gómez
Ing. Rolén Coto Calderón

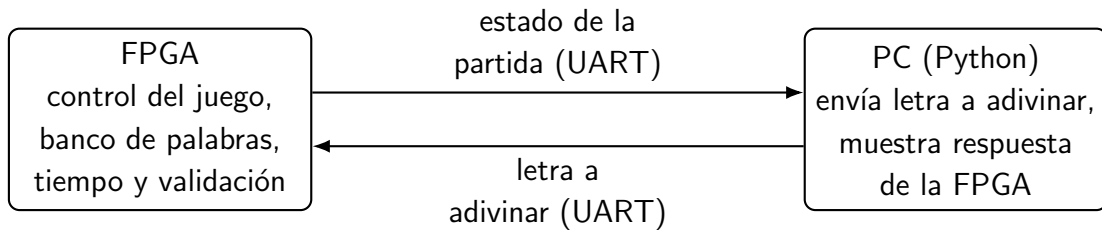
1. Introducción

Este segundo proyecto integra diseño digital secuencial, arquitectura modular de periféricos y comunicación serial para construir una aplicación interactiva completa entre la FPGA y una computadora personal. A diferencia del Proyecto 1, donde el control del juego dependía únicamente de entradas locales (*push buttons*), en este proyecto la FPGA debe coordinar su lógica de control con una aplicación externa que se ejecuta en la PC, comunicándose con ella mediante un periférico UART y presentando el estado del juego de forma local en una pantalla LCD y en displays de 7 segmentos.

La aplicación a desarrollar es el clásico juego de adivinanza de palabras *Ahorcado*. A diferencia de la mayoría de implementaciones de este juego, aquí **la FPGA es quien concentra toda la inteligencia y el control de la partida**: selecciona la palabra secreta de forma pseudoaleatoria a partir de un banco interno, valida cada letra recibida, controla el tiempo restante y determina el resultado de la partida. La PC actúa únicamente como una terminal remota para el jugador: una aplicación sencilla en Python que envía la letra que el usuario desea adivinar y despliega la respuesta que la FPGA le devuelve.

Indicaciones generales:

- Lea y trate de comprender todo el trabajo solicitado antes de iniciarlo.
- Durante el desarrollo del diseño y la validación, asegúrese de documentar todas las tablas de verdad, circuitos, figuras, diagramas (flujo, estado, bloques, tiempos), entre otros, que sean necesarios. Esto será **fundamental** para la revisión del diseño y para la documentación del proyecto. Siga los lineamientos del diseño modular.



LCD, displays 7 seg,
LED de estado, buzzer,
push buttons (modo/reinicio)

Figura 1: Arquitectura general del sistema: la FPGA concentra el control completo de la partida; la PC es una terminal de entrada/salida remota conectada por UART.

- Para la presentación funcional, se le pedirá que muestre algunos de los diseños propuestos, ya sea en implementación o simulación. Dicha selección se hará el día de la presentación.
- Todos los bloques de diseño deben ser **sintetizables** en SystemVerilog. El sistema debe funcionar con un único reloj de entrada de 100 MHz.

Los sistemas desarrollados en este proyecto deben ser completamente **sintetizables**, descritos utilizando diseño RTL con SystemVerilog. Tome en cuenta que estará desarrollando, como parte de su solución, lógica combinacional y secuencial, por lo que los bloques desarrollados deben estar libres de elementos de memoria no intencionados, especialmente *latches*. Compruebe, por medio de los mensajes, visualización del *netlist* de la herramienta, y la utilización del *linter*, que no se generen dichos elementos como parte del proceso de síntesis. Las simulaciones deben ser presentadas a nivel de **post-implementación** temporizadas.

2. Investigación previa

1. Investigue el funcionamiento general de módulos LCD compatibles con controlador HD44780 y su secuencia de inicialización.
2. Investigue la documentación del PmodCLP: https://digilent.com/reference/_media/pmod:pmod:pmodclp_rm.pdf
3. Investigue el protocolo UART: estructura de trama (*start*, datos, paridad opcional, *stop*), relación entre reloj y *baud rate*, y consideraciones de validación para transmisión/recepción confiable en FPGA.
4. Investigue buenas prácticas para diseño de periféricos mapeados a memoria con buses de 32 bits.
5. Investigue técnicas para el almacenamiento de bancos de datos constantes (por ejemplo, tablas de cadenas de texto) en FPGA mediante memorias de solo lectura (ROM) sintetizables descritas en SystemVerilog, incluyendo estrategias de codificación de cadenas de longitud variable en palabras de ancho fijo.

6. Investigue la generación de números pseudoaleatorios mediante registros de desplazamiento con retroalimentación lineal (LFSR), retomando lo estudiado en el Proyecto 1, y cómo aplicar dicho concepto, ahora descrito completamente en HDL sintetizable, para seleccionar un índice dentro de un banco de datos.
7. Investigue técnicas de diseño de temporizadores regresivos (*countdown timers*) y su despliegue en displays de 7 segmentos multiplexados.
8. Investigue técnicas de *debouncing* para pulsadores y su importancia en interfaces de selección de modo mediante *push buttons*.

3. Especificaciones

3.1. Aplicación del juego: Ahorcado

- **Todo el control del juego debe implementarse en la FPGA:** selección de la palabra secreta, validación de letras, conteo de tiempo e intentos, y determinación del resultado de la partida. La PC se considera una terminal de entrada/salida por UART y no debe contener lógica de control del juego.
- El sistema debe contener un banco de **al menos 50 palabras distintas**, de longitud variable, almacenado en una memoria de solo lectura (ROM) sintetizable dentro de la FPGA. Las palabras deben usar únicamente letras A--Z (sin tildes ni ñ), con longitud entre 4 y 12 caracteres.
- Al inicio de cada partida, la FPGA debe seleccionar una palabra del banco de forma **pseudoaleatoria**, mediante un generador tipo LFSR descrito en SystemVerilog.
- Cada partida tiene un **límite de tiempo** para adivinar la palabra completa, mostrado en cuenta regresiva en los displays de 7 segmentos. Si el tiempo llega a cero antes de completar la palabra, la partida se pierde.
- Cada partida admite un máximo de **6 letras incorrectas** (siguiendo la convención clásica del ahorcado). Al alcanzar la sexta letra incorrecta, la partida se pierde, independientemente del tiempo restante.
- Si la letra recibida coincide con una o más posiciones de la palabra secreta, **todas** las posiciones correspondientes deben revelarse simultáneamente en el LCD.
- Una letra ya adivinada previamente (correcta o incorrecta) que se reciba de nuevo debe **ignorarse**: no debe consumir un intento fallido ni reiniciar el temporizador.
- La partida se gana cuando todas las letras de la palabra secreta han sido reveladas antes de agotar el tiempo o los intentos.

3.2. Selección de modo de dificultad

Antes de iniciar cada partida, el sistema debe presentar en el LCD una pantalla de selección de modo de dificultad:

- **BTN_SEL**: alterna cíclicamente la opción mostrada en el LCD entre FÁCIL y DIFÍCIL.
- **BTN_OK**: confirma el modo actualmente mostrado e inicia la partida.
- **Modo Fácil**: la palabra secreta puede ser cualquiera del banco (longitud mínima de 4 letras). Límite de tiempo sugerido: 60 s.
- **Modo Difícil**: la palabra secreta se selecciona únicamente entre aquellas con **más de 5 letras** (6 o más) del banco. Límite de tiempo sugerido: 45 s.

Cada equipo debe documentar y justificar los valores exactos de tiempo utilizados para cada modo, siempre que se mantenga la relación de que el modo difícil ofrece menos tiempo que el fácil. Al finalizar una partida (por victoria, derrota por intentos o derrota por tiempo), el sistema debe regresar automáticamente a la pantalla de selección de modo para iniciar una nueva partida.

3.3. Flujo de una partida

Cada partida debe seguir el flujo mínimo:

1. Mostrar la pantalla de selección de modo y esperar confirmación (BTN_OK).
2. Seleccionar una palabra del banco acorde al modo elegido mediante el generador pseudo-aleatorio.
3. Iniciar el temporizador de cuenta regresiva y mostrar en el LCD la palabra oculta (todas las posiciones como guiones bajos o carácter equivalente) junto con el conteo de intentos fallidos disponibles.
4. Esperar la recepción de una letra por UART desde la aplicación de PC.
5. Validar la letra recibida contra la palabra secreta y actualizar el LCD: revelar las posiciones correspondientes si la letra es correcta, o decrementar los intentos disponibles si es incorrecta. Enviar por UART a la PC el resultado de la validación y el estado actualizado de la palabra.
6. Repetir los pasos 4 y 5 hasta que la palabra se complete, se agoten los intentos, o se agote el tiempo.
7. Mostrar el resultado final (GANÓ / PERDIÓ) en el LCD durante al menos 3 s, actualizar el contador acumulado de partidas ganadas, y regresar a la pantalla de selección de modo.

El diseño debe definir explícitamente, y documentarlo, qué ocurre si se recibe una letra por UART mientras el sistema se encuentra en la pantalla de selección de modo o mostrando el resultado final (por ejemplo, ignorar dicha entrada).

3.4. Periféricos

3.4.1. Periférico LCD (diseño del equipo)

Cada grupo debe diseñar su propio periférico para controlar el PmodCLP (LCD 16x2 compatible HD44780). Este periférico debe presentar interfaz de **32 bits** y, como mínimo, los siguientes dos registros:

- **Registro 0: CONTROL/ESTADO** (offset 0x00, 32 bits)
- **Registro 1: DATOS** (offset 0x04, 32 bits)

Definición mínima de campos:

- Registro CONTROL/ESTADO:
 - bit 0: `start` (W1P). Inicia la transacción LCD usando la configuración actual; se limpia automáticamente.
 - bit 1: `rs` (0=comando, 1=dato). Define si `data_byte` se interpreta como instrucción o carácter.
 - bit 2: `clear` (W1P). Solicita limpiar pantalla y retornar DDRAM a estado inicial.
 - bit 3: `home` (W1P). Solicita llevar cursor a posición de inicio sin borrar contenido.
 - bit 8: `busy` (RO). Indica periférico ocupado; mientras esté en 1 no debe aceptarse una nueva operación.
 - bit 9: `done` (RO). Pulso/flag de finalización de la última operación LCD aceptada.
 - bits [31:16]: reservado/uso del equipo (documentar).
- Registro DATOS:
 - bits [7:0]: `data_byte` (ASCII o código de instrucción). Carga útil enviada al LCD cuando se activa `start`.
 - bits [31:8]: reservado/uso del equipo (documentar).

Requisitos funcionales del periférico LCD:

- Inicialización correcta del LCD al arranque.
- Soporte de escritura de caracteres.
- Soporte de comandos de limpieza y retorno a inicio.
- Manejo de temporización interna acorde al dispositivo (no se permite asumir escrituras instantáneas).
- Señalización de estado (`busy/done`) para sincronización con la lógica de control del juego.

3.4.2. Periférico UART

Para este proyecto se proveerá un **módulo UART base funcional** (núcleo TX/RX). A partir de ese núcleo, cada equipo debe diseñar el **periférico UART**, que es el bloque encargado de exponer la interfaz de registros y la lógica de control hacia el sistema. El periférico UART diseñado por el equipo debe integrarse correctamente y demostrar comunicación bidireccional confiable con la PC. El acceso por registros debe realizarse mediante la interfaz estándar definida en la subsección siguiente.

- El periférico UART debe considerar un registro de control y un par de registros para datos de entrada y salida.
- Registro CONTROL:
 - bit 0: `send` (WC). Un valor en alto en este bit dispara una transferencia de salida por el puerto serie. El bit se mantiene en alto durante el proceso y se baja a cero, automáticamente, al finalizar la operación. Durante una transferencia, el sistema envía el dato almacenado en el registro de datos 0 (`addr_i[1:0]=2'b00`). Al finalizar la transacción, la unidad de control interna debe limpiar el bit `send` para indicar el final.
 - bit 1: `new_rx` (RW). Este bit indica que la interfaz UART ha recibido un byte desde el sistema externo conectado a la FPGA. Cada vez que la interfaz UART recibe un dato, debe almacenarlo en el registro de datos 1 (`addr_i[1:0]=2'b01`). Para indicar que se ha recibido un nuevo dato, la unidad de control interna debe colocar un valor de 1 en `new_rx`. Es responsabilidad del bloque que instancia la interfaz limpiar `new_rx` luego de haber leído el registro de datos 1.
- Registros DATOS:
 - El periférico UART posee dos registros de 32 bits cada uno. Para escribir o leer estos registros se utiliza `addr_i[1:0]` de la interfaz estándar.
 - Cada registro posee un único campo llamado `DAT0`, de 8 bits, `[7:0]`, el cual es de lectura y escritura.
 - En el caso del registro 0, el dato presente en este campo es enviado por la interfaz UART cuando se detecta un uno en el bit `send`.
 - El registro 1 es utilizado en la recepción de datos. El campo `DAT0` reflejará el dato recibido de forma asíncrona.
- La interfaz UART debe operar a 115200 baudios.
- Desarrolle un bloque de pruebas apropiado que permita enviar a una computadora personal valores conocidos, con lo que pueda validar el funcionamiento del periférico UART.

3.4.3. Interfaz estándar de periféricos

Para los periféricos del proyecto (al menos UART y LCD), se debe usar una interfaz digital común de 32 bits para facilitar integración y reutilización. La interfaz mínima se define así:

Señal	Dirección	Descripción
clk_i	Entrada	Reloj del sistema.
rst_i	Entrada	Reinicio síncrono o asíncrono, según defina el equipo y documento.
write_enable_i	Entrada	Indica escritura cuando está en 1.
addr_i[1:0]	Entrada	Dirección de registro interno (00, 01, 10, 11).
wdata_i[31:0]	Entrada	Dato de escritura hacia el periférico.
rdata_o[31:0]	Salida	Dato leído desde el periférico en la dirección seleccionada por addr_i[1:0].

Notas de uso:

- Cuando write_enable_i=1, el periférico escribe wdata_i[31:0] en el registro seleccionado por addr_i[1:0].
- Cuando write_enable_i=0, la lectura se realiza por rdata_o[31:0] desde el registro seleccionado por addr_i[1:0].
- Los registros deben mapearse con addr_i[1:0] y documentarse claramente.

3.4.4. Protocolo de aplicación sobre UART

Sobre el enlace UART, además de la interfaz de registros del periférico, cada equipo debe definir y documentar un protocolo de aplicación mínimo con los siguientes mensajes:

- **PC → FPGA:** un byte ASCII con la letra mayúscula (A--Z) que el usuario desea adivinar. Cualquier byte recibido que no corresponda a una letra mayúscula debe ser descartado por la FPGA sin afectar la partida en curso; la aplicación de PC debe además validar la entrada antes de transmitirla.
- **FPGA → PC:** al menos los siguientes eventos deben notificarse a la PC: inicio de partida (incluyendo la longitud de la palabra y el modo seleccionado), resultado de cada letra recibida (acierto/error y patrón actualizado de la palabra), e intentos fallidos restantes, y resultado final de la partida (victoria o derrota y causa). El formato exacto de trama (delimitadores, longitud, orden de campos) queda a criterio del equipo, siempre que sea consistente y esté completamente documentado.
- El tiempo restante **no** necesita transmitirse por UART: se muestra únicamente de forma local en los displays de 7 segmentos de la tarjeta.

3.4.5. Indicadores y controles locales

- **Displays de 7 segmentos:** utilice al menos 4 dígitos. Se sugiere asignar 2 dígitos a la cuenta regresiva de tiempo restante (en segundos) y 2 dígitos al contador acumulado de partidas ganadas (00–99) desde el último reinicio general del sistema.

- **LED de estado:** indique con al menos un LED si el sistema se encuentra en la pantalla de selección de modo, en una partida activa, o mostrando el resultado final, de forma claramente distinguible.
- **Buzzer:** genere una retroalimentación sonora distinta para letra correcta, letra incorrecta, y fin de partida (victoria/derrota).
- **Botón central** (BTN_RST): reinicia el sistema en cualquier momento, regresando a la pantalla de selección de modo y restableciendo el contador de partidas ganadas.

3.5. Aplicación de PC en Python

Como parte de la especificación del sistema, se debe implementar una aplicación en Python para la PC que actúe como interfaz del jugador remoto mediante UART. Esta aplicación debe:

- solicitar al usuario una letra (A--Z) y transmitirla a la FPGA por UART,
- mostrar el estado recibido desde la FPGA: patrón actual de la palabra, intentos fallidos restantes, y resultado de la última letra enviada,
- mostrar el resultado final de la partida cuando la FPGA lo notifique,
- manejar y reportar entradas inválidas (por ejemplo, más de un carácter, símbolos no alfabéticos) sin afectar la estabilidad de la partida ni bloquear la aplicación.

3.6. Arquitectura del sistema

La arquitectura interna del sistema queda **abierta** para que cada equipo proponga su diseño. Puede implementarse con la partición modular y estrategia de control que consideren más adecuada, siempre que se cumplan todas las especificaciones funcionales, de interfaz y de validación establecidas en este documento.

4. Entregables

1. Planteamiento del diseño:

- propuesta de arquitectura modular con enfoque **top-down**, siguiendo los principios de diseño modular;
- objetivo de cada diagrama/módulo y rol dentro del sistema,
- definición de entradas, salidas y señales internas relevantes por bloque principal,
- entrega del planteamiento en docs/planteamiento.md (o archivo equivalente dentro de docs/).

2. Funcionalidad final:

- demostración de una partida completa en cada modo de dificultad (fácil y difícil),
- evidencia de comunicación bidireccional con la aplicación de PC,
- evidencia del límite de tiempo y del límite de intentos fallidos operando correctamente,
- evidencia de visualización de palabra, intentos y tiempo restante en LCD y displays de 7 segmentos,
- evidencia de retroalimentación sonora (buzzer) para aciertos, errores y fin de partida,
- simulación **post-implementación** temporizada del sistema, cubriendo al menos la recepción y validación de una letra,
- evidencia de **tests de autochequeo** asociados a la funcionalidad principal.

3. Documentación técnica en formato Markdown dentro de docs/ (por ejemplo, docs/informe.md):

- como referencia de formato para la documentación técnica, puede consultar este ejemplo,
- aspectos teóricos relevantes para sustentar el diseño propuesto,
- enfoque de la solución (decisiones de diseño y justificación técnica),
- descripción formal de interfaces de módulos,
- definición del periférico LCD y del protocolo de aplicación sobre UART (registros, bits, tramas, semántica),
- diagramas de estado de control principal y periférico LCD,
- estrategia de validación (simulación y pruebas en FPGA),
- evidencia de simulaciones con pruebas de autochequeo (criterios de pase/fallo y resultados),
- resultados y evidencias funcionales (capturas, tablas y observaciones), incluyendo uso de recursos y análisis de *timing*,
- breve análisis crítico de lo realizado (logros, limitaciones y mejoras posibles),
- análisis de problemas encontrados y cómo se resolvieron.

4. Repositorio y trazabilidad:

- código fuente HDL y *constraints*,
- aplicación Python de PC para interfaz UART,
- bancos de prueba y **tests de autochequeo**,
- scripts de simulación/síntesis (si aplica),
- README.md principal con explicación clara de la estructura del proyecto/repositorio,
- historial de trabajo con *commits* suficientes y descriptivos,
- uso de flujo de desarrollo moderno con **issue tracking** (incluyendo asignación de tareas por integrante), **feature branches**, **commits significativos** y uso de **merge/pull requests** entre integrantes del equipo, siguiendo *git workflow*.

5. Evaluación

Este proyecto tiene una duración de tres semanas a partir de su inicio. El planteamiento del diseño debe ser validado con el profesor **previo a la presentación funcional**. Ningún proyecto será evaluado sin el visto bueno desde la perspectiva de diseño.

Las fechas oficiales de este proyecto son:

- **Inicio:** G1: 27.08.2026 / G2: 26.08.2026
- **Planteamiento del diseño:** G1: 03.09.2026 / G2: 02.09.2026
- **Entrega del informe técnico:** G1: 17.09.2026 / G2: 14.09.2026
- **Fin:** G1: 17.09.2026 / G2: 14.09.2026

El presente proyecto tiene un valor del **15 %** de la nota final del curso y se evaluará mediante cuatro rubros, cada uno calificado con una rúbrica común a todos los proyectos del curso. El detalle de cada rúbrica se presenta en el Anexo A.

1. Diseño	20 %
2. Presentación funcional	50 %
3. Informe técnico	15 %
4. Repositorio	15 %

El formato de entrega de cada documento (Markdown enriquecido, ubicado en la carpeta correspondiente del repositorio) se detalla junto a su rúbrica en el Anexo A.

A. Anexo: Rúbricas de evaluación

Las siguientes rúbricas son comunes a todos los proyectos del curso y se detallan a continuación para que el estudiante cuente con los criterios de evaluación completos dentro de este mismo documento.

A.1. Diseño

Documentación del diseño

Rúbrica aplicable al planteamiento del diseño entregado en cada proyecto del curso. El diseño debe seguir la metodología de diseño modular vista en clase. Evalúa la calidad de los diagramas, la justificación técnica de las decisiones tomadas, la completitud del documento y la claridad general de la presentación.

Formato de entrega: el planteamiento del diseño debe entregarse como un archivo Markdown (.md) enriquecido, ubicado en la carpeta docs/diseño/ del repositorio del proyecto. No se aceptan formatos alternativos, como un PDF exportado desde un procesador de texto externo, como entrega oficial.

Nivel	Descripción del criterio
Diagramas y representaciones del diseño (peso: 30 %)	
Excelente	Diagramas claros, correctos y bien organizados (bloques, flujo, estados, esquemas u otros). Reflejan correctamente la arquitectura del sistema y facilitan la comprensión.
Satisfactorio	Diagramas mayormente correctos, pero con detalles poco claros, incompletos o escasa explicación.
Regular	Diagramas simplificados o con errores que dificultan entender el diseño.
Deficiente	No hay diagramas o son incorrectos/confusos.
Justificación y decisiones de diseño (peso: 30 %)	
Excelente	Explica y justifica claramente las decisiones de diseño (arquitectura, señales, ecuaciones, implementación, etc.). Demuestra razonamiento técnico sólido.
Satisfactorio	Presenta las decisiones principales, pero con justificación limitada.
Regular	Justificación mínima o superficial de las decisiones de diseño.
Deficiente	No se explican ni justifican las decisiones de diseño.
Completitud del documento de diseño (peso: 20 %)	
Excelente	Incluye todos los elementos solicitados (objetivos, tablas, ecuaciones, diagramas, estrategia de implementación, plan de pruebas, etc.) de forma completa y bien estructurada.

Nivel	Descripción del criterio
Satisfactorio	Incluye la mayoría de los elementos requeridos, con pequeños faltantes o secciones poco desarrolladas.
Regular	Varias secciones importantes faltan o están incompletas.
Deficiente	El documento es muy incompleto o no sigue la estructura solicitada.
Claridad y calidad de la documentación (peso: 20 %)	
Excelente	Documento bien organizado, claro y profesional. Uso adecuado de Markdown, tablas, figuras y notación técnica.
Satisfactorio	Documento comprensible pero con problemas menores de organización o presentación.
Regular	Organización deficiente o redacción poco clara.
Deficiente	Documento difícil de leer o mal presentado.

A.2. Presentación funcional

Presentación funcional — Proyecto 2

Rúbrica aplicable a la demostración funcional del Proyecto 2. Evalúa el funcionamiento de cada etapa esperada del sistema, implementado en su totalidad en la FPGA y comunicado con la aplicación de PC mediante UART, la calidad de la verificación por simulación de la parte descrita en SystemVerilog, y la defensa técnica del diseño ante preguntas del profesor durante la presentación.

Nivel	Descripción del criterio
Periférico LCD (peso: 15 %)	
Excelente	El periférico LCD funciona correctamente en todas las pantallas del sistema (selección de modo, palabra oculta/revelada, resultado final), sin errores de inicialización ni parpadeos, actualizando el contenido de forma oportuna y sincronizada con el estado real del juego.
Satisfactorio	El LCD funciona correctamente en la mayoría de los casos, con fallas ocasionales de refresco o inicialización que no impiden seguir la partida.
Regular	El LCD presenta fallas frecuentes (caracteres incorrectos, refresco lento, pantallas incompletas) que dificultan seguir el estado del juego.
Deficiente	El LCD no funciona o no muestra información reconocible del estado del juego.
Comunicación serial (UART) con la PC (peso: 15 %)	
Excelente	La comunicación UART entre la FPGA y la aplicación de PC funciona de forma confiable, con un protocolo de aplicación bien definido y documentado, sin pérdida ni corrupción de tramas, y manejo correcto de entradas inválidas sin afectar la estabilidad de la partida.

Nivel	Descripción del criterio
Satisfactorio	La comunicación funciona en la mayoría de los casos, con errores ocasionales de sincronización o pérdida esporádica de datos que no impiden continuar la demostración.
Regular	La comunicación presenta errores frecuentes que afectan visiblemente el desarrollo de la partida, aunque se demuestra transmisión de datos en ambos sentidos.
Deficiente	No existe comunicación funcional entre la FPGA y la PC, o los datos recibidos/enviados no corresponden al protocolo especificado.
Manejo de entradas/salidas locales (peso: 5 %)	
Excelente	Los botones BTN_SEL, BTN_OK y BTN_RST responden correctamente y libres de rebotes; los displays de 7 segmentos, el LED de estado y el buzzer reflejan en todo momento el estado real del juego.
Satisfactorio	Las entradas y salidas funcionan correctamente en la mayoría de los casos, con fallas ocasionales de rebote, sincronización o actualización de displays/buzzer.
Regular	Existen fallas evidentes — rebotes no filtrados, displays incorrectos, LED o buzzer que no reflejan la situación real — que afectan la jugabilidad.
Deficiente	Las entradas o salidas no funcionan de forma reconocible, o el sistema no responde a las pulsaciones.
Control del juego (banco de palabras, dificultad y temporización) (peso: 25 %)	
Excelente	La lógica de control selecciona correctamente la palabra mediante el generador LFSR según el modo elegido (fácil/difícil), aplica el límite de tiempo y el número máximo de intentos fallidos especificado para cada modo, revela correctamente todas las ocurrencias de una letra acertada, e ignora letras repetidas sin penalizar, determinando de forma correcta el resultado final de la partida.
Satisfactorio	La lógica de control funciona correctamente en la mayoría de los casos, con desviaciones menores respecto a los tiempos, longitudes de palabra o número de intentos especificados por modo.
Regular	La lógica de control presenta fallas evidentes en la selección de la palabra, el manejo de la dificultad, o el conteo de tiempo o de intentos, aunque el juego es parcialmente jugable.
Deficiente	La lógica de control no funciona de forma reconocible, o el juego no sigue las reglas especificadas.
Simulación autoverificable (FPGA/SystemVerilog) (peso: 15 %)	
Excelente	Presenta simulaciones autoverificables (<i>testbenches</i> con verificación automática de resultados) para los bloques principales descritos en SystemVerilog, con reporte claro de aciertos/fallos y cobertura de los casos relevantes, incluyendo condiciones de borde.

Nivel	Descripción del criterio
Satisfactorio	Presenta simulaciones autoverificables para la mayoría de los bloques, aunque con cobertura limitada de casos de borde o verificación parcialmente manual.
Regular	Las simulaciones presentadas requieren inspección manual de formas de onda para determinar su correctud; cobertura escasa de los bloques del diseño.
Deficiente	No presenta simulaciones autoverificables, o las presentadas no permiten verificar el comportamiento correcto del diseño.
Defensa y comprobación de dominio técnico (peso: 25 %)	
Excelente	Responde con precisión y seguridad a las preguntas técnicas sobre el diseño, justificando decisiones de arquitectura e implementación, e identificando con claridad el funcionamiento de cada bloque ante modificaciones o pruebas solicitadas por el profesor durante la presentación.
Satisfactorio	Responde correctamente a la mayoría de las preguntas técnicas, aunque con dudas puntuales o explicaciones incompletas sobre algunos bloques del diseño.
Regular	Responde de forma superficial o memorística, con dificultad para justificar decisiones de diseño o para explicar el funcionamiento ante cambios solicitados.
Deficiente	No logra explicar ni justificar el funcionamiento del diseño, o las respuestas evidencian que no comprende su propia implementación.

A.3. Informe técnico

Documentación técnica (informe)

Rúbrica aplicable al informe técnico final entregado en cada proyecto del curso. Evalúa la fundamentación teórica, la presentación y el análisis de los resultados, las conclusiones obtenidas y la calidad general del documento.

Formato de entrega: el informe técnico debe entregarse como un archivo Markdown (.md) enriquecido, ubicado en la carpeta docs/informe/ del repositorio del proyecto. No se aceptan formatos alternativos, como un PDF exportado desde un procesador de texto externo, como entrega oficial.

Nivel	Descripción del criterio
Fundamentación teórica (peso: 20 %)	
Excelente	Presenta los conceptos teóricos relevantes con claridad y correcta relación con el laboratorio/proyecto. Integra adecuadamente el marco teórico con el trabajo realizado.
Satisfactorio	Presenta la mayoría de los conceptos necesarios, aunque con poca profundidad o conexión limitada con el laboratorio/proyecto.

Nivel	Descripción del criterio
Regular	Marco teórico superficial o incompleto.
Deficiente	No presenta fundamentación teórica o es incorrecta.
Presentación de resultados (peso: 30 %)	
Excelente	Resultados completos y organizados (tablas, gráficas, simulaciones, mediciones). Claramente etiquetados y bien documentados.
Satisfactorio	Resultados mayormente completos, aunque con organización o explicación limitada.
Regular	Resultados incompletos o presentados de forma poco clara.
Deficiente	No presenta resultados relevantes o son incorrectos.
Análisis e interpretación de resultados (peso: 25 %)	
Excelente	Analiza los resultados de forma crítica, compara valores teóricos, simulados y experimentales cuando aplica, e identifica causas de diferencias o errores.
Satisfactorio	Presenta análisis general de los resultados, pero con poca profundidad o discusión limitada.
Regular	Análisis superficial, descriptivo o con escasa interpretación técnica.
Deficiente	No analiza los resultados o las conclusiones no se basan en los datos.
Conclusiones y aprendizaje obtenido (peso: 15 %)	
Excelente	Conclusiones claras, bien fundamentadas y directamente relacionadas con los resultados y objetivos del laboratorio/proyecto. Incluye reflexiones o lecciones aprendidas.
Satisfactorio	Conclusiones correctas pero generales o poco conectadas con los resultados.
Regular	Conclusiones superficiales o parcialmente relacionadas con el trabajo realizado.
Deficiente	No presenta conclusiones o son incorrectas.
Calidad y organización del documento (peso: 10 %)	
Excelente	Documento claro, bien estructurado y profesional. Uso adecuado de Markdown, figuras, tablas, ecuaciones y referencias.
Satisfactorio	Organización adecuada con pequeños problemas de claridad o formato.
Regular	Documento desorganizado o difícil de seguir.
Deficiente	Documento mal estructurado o incompleto.

A.4. Repositorio

Gestión de repositorio

Rúbrica aplicable al repositorio Git de cada proyecto del curso. Evalúa el historial de commits, el uso de ramas de desarrollo, la gestión de issues, la revisión colaborativa de cambios y la estructura

del README.

Nivel	Descripción del criterio
Historial de commits y calidad de código (peso: 20 %)	
Excelente	Commits frecuentes, claros y descriptivos, reflejan avances reales del proyecto y buena trazabilidad. Código documentado y organizado correctamente.
Satisfactorio	Commits suficientes pero con mensajes genéricos o frecuencia irregular. Código sin comentar o desordenado.
Deficiente	Pocos commits, sin descripción clara o sin reflejar el desarrollo. No tiene código real.
Uso de ramas de desarrollo (peso: 20 %)	
Excelente	Uso sistemático de ramas por funcionalidad o integrante, siguiendo un flujo profesional (Git Flow o similar).
Satisfactorio	Se usan ramas ocasionalmente o sin convención clara.
Deficiente	Todo el trabajo se hace en una sola rama sin control de versiones adecuado.
Gestión de issues y seguimiento (peso: 20 %)	
Excelente	Issues bien definidos y organizados, usados para planificar tareas, reportar bugs y documentar avances.
Satisfactorio	Uso limitado de issues o sin seguimiento completo.
Deficiente	No se usan issues o no hay evidencia de planeamiento.
Revisión colaborativa (pull/merge requests) (peso: 20 %)	
Excelente	Uso activo de pull o merge requests con revisiones cruzadas entre miembros antes de integrar cambios.
Satisfactorio	Uso ocasional o sin revisión efectiva.
Deficiente	No se utilizan revisiones ni solicitudes de integración.
Estructura del README (peso: 20 %)	
Excelente	README completo y organizado: incluye propósito, dependencias, instalación, compilación y ejecución.
Satisfactorio	README con información parcial o poco detallada.
Deficiente	README ausente o sin instrucciones útiles.